Data Science Capstone Project Report

SpaceX Falcon 9 First-Stage Landing Prediction

Project type: Exploratory data analysis and supervised machine learning

Data source: SpaceX public API v4 — https://api.spacexdata.com/v4/

Status: Reproducible project draft; final empirical results require running the notebooks.

Executive summary

This project examines whether public launch metadata can help characterize and predict successful Falcon 9 first-stage landings. The workflow collects launch and related metadata, checks data quality, explores observed landing outcomes by time and launch site, and compares baseline classification algorithms. The goal is to demonstrate an end-to-end data-science workflow rather than make operational flight decisions.

This document is a methodology-first draft. Because the data-collection and modeling notebooks have not been executed in this generated package, it intentionally does not state fabricated sample counts, accuracy scores, confusion-matrix values, or a winning model. Replace this paragraph with measured findings after executing the workflow.

1. Business understanding

SpaceX's reusable first stage is central to reducing launch costs. A data-science model that estimates landing outcomes from available mission metadata can support retrospective analysis and help identify patterns worth investigating. However, observational data cannot establish causality, and a model trained on historical launches should not be used for safety-critical decisions without extensive validation.

Objective

Build and evaluate a supervised classifier for the known first-stage landing outcome using selected launch metadata.

Target definition

The target is `landing_success`, sourced from the first listed core's `landing_success` field in the API. Unknown outcomes are treated as missing and excluded from supervised training. Upcoming launches are excluded.

2. Data collection

The project retrieves launch records, rocket metadata, launchpad metadata, payload metadata, and core metadata from the public SpaceX API v4. These sources are flattened into one row per launch. Fields include flight number, date, rocket, launch site, payload mass, orbit, core reuse, landing attempt, and landing success.

Because this is a live API, the dataset can change over time. Record the retrieval date and preserve the exact CSV used for the final analysis.

3. Data wrangling

The pipeline:

• converts launch dates to UTC datetimes and extracts year/month;

• flattens rocket, launchpad, payload, and core metadata;

• aggregates payload count and total known payload mass;

• retains unknown landing outcomes as missing in the raw data;

• maps known Boolean landing outcomes to 0/1 for modeling.

Missingness should be examined by column. Missing data can mean “not recorded” or “not applicable”; it should not automatically be imputed as failure.

4. Exploratory data analysis

The EDA notebook creates:

• counts of known successful and unsuccessful landing outcomes;

• year-level success-rate summaries;

• launch-site success-rate summaries with a minimum group-size threshold;

• SQL summaries using SQLite.

When interpreting charts, consider changing launch cadence, vehicle variants, mission types, launch sites, and data availability. Small groups can show 0% or 100% success rates by chance. Correlation does not establish a causal effect.

5. Modeling methodology

The baseline comparison includes Logistic Regression, Decision Tree, Support Vector Machine, and K-Nearest Neighbors. Numeric variables are median-imputed and standardized; categorical variables are imputed and one-hot encoded. A stratified train/test split is used for an initial comparison.

Leakage and validity

Before finalizing the feature set, confirm every feature would be known at the intended prediction time. Landing-attempt and landing-outcome fields must not be predictors. The starter feature set includes historical metadata such as flight number and launch year, which may encode temporal patterns; justify these features or remove them. A chronological split is recommended as a robustness check.

6. Evaluation

The notebook reports accuracy, precision, recall, F1, ROC-AUC where defined, and a confusion matrix. Report the test-set size and target class distribution beside the scores. Accuracy alone is not enough if the classes are imbalanced. Select a model based on the project objective and the relative cost of false positives and false negatives, not merely the largest single metric.

Results table: generated at `outputs/model_metrics.csv` after running notebook 03.

Confusion matrix: generated at `outputs/confusion_matrix.png` after running notebook 03.

No measured performance values are included here because the notebooks have not been run in this package.

7. Dashboard

An optional Streamlit dashboard summarizes the number of launch records, known landing outcomes, observed landing success rate, and site-level descriptive rates. It is intended for exploration, not operational decision-making.

Run:

```
streamlit run dashboard/app.py
```

8. Limitations

• The public API may change and may contain missing or revised records.

• Historical outcomes are observational and may be confounded by time, vehicle configuration, mission type, and launch site.

• Small groups can produce unstable success-rate estimates.

• Random train/test splitting can overstate generalization when records are temporally related.

• A metadata-only classifier may omit important operational conditions.

• This project is educational and is not suitable for safety-critical launch decisions.

9. Conclusion and future work

This capstone scaffold provides a repeatable path from data acquisition to cleaning, EDA, SQL, classification, and visualization. After execution, conclusions should be grounded in the generated metric table, confusion matrix, and error analysis. Potential extensions include a time-based test set, cross-validation, calibration analysis, feature ablation, uncertainty estimates, and a careful comparison of models with and without temporal features.

Reproducibility checklist

■ Record dataset retrieval date and API source.

■ Run notebooks 01, 02, and 03 from top to bottom.

■ Inspect data quality and target balance.

■ Confirm there is no target leakage.

■ Save the final metric table and confusion matrix.

■ Update the executive summary with actual measured findings.

■ Add screenshots / dashboard outputs only after they are generated.

■ Add the actual GitHub repository URL after creating the repository.